

Tiny YOLO Optimization Oriented Bus Passenger Object Detection

ZHANG Shuo, WU Yanxia, MEN Chaoguang and LI Xiaosong

(College of Computer Science and Technology, Harbin Engineering University, Harbin 150001, China)

Abstract — The real-time collection of bus passenger object detection is an essential part of developing a smart bus system. The difficulty of object detection mainly lies in the objective factors, such as: clothing, hair style and accessories, light, *etc.* Traditional object detection methods with the artificial feature extraction suffers from insufficient strength in expression, generalization, and recognition rate. The object detection method based on deep learning mainly uses the convolutional neural network in deep learning to learn features from a large set of data. The learned features can describe the rich information inherent in the data, and improve the expression ability of the features as well as the recognition accuracy. Due to too many parameters of the Convolutional neural network (CNN) model, the amount of calculation is too large to be operated on the vehicle terminal. To reduce calculation burden and improve the operation speed, we employs the depthwise separable convolution method to optimize the convolutional layer of tiny YOLO network model. It decomposes a complete convolution operation into depthwise convolution and pointwise convolution, thus reducing the parameter amount of the CNN and improving the operation speed. The experiment results reveal that the speed of bus passenger object detection detected by our improved model is 4 times faster than the previous one but with the nearly same detection accuracy.

Key words — Bus system, Object detection, Model optimization, Tiny YOLO.

I. Introduction

With the continuous advancement of the science and technology, the smart public transport system has developed rapidly in recent years. The bus passenger object detection plays an important role in the entire smart bus system. By performing statistical analysis, it can provide accurate reference information for the dispatch of the vehicle, thereby optimizing the allocation

of bus resources, improving the overall running speed of the bus system and the passenger travel satisfaction.

In the past, the bus system depended on bus passenger object detection through pressure detection. The pressure pedal is pressed by the passenger's foot to sense the presence of a person. This method is low in cost, but it is not accurate to identify the situation where the pressure pedal is pressed simultaneously by many feet. Another detection method is based on the infrared detection. The infrared light of a specific wavelength emits an infrared sensor installed above the door and is re-accepted after being reflected by the human body. By analyzing and processing the received data, the number of passengers can be calculated. This method is only suitable for passengers with clear separations, but it is not ideal for passengers with overlapping crowds during peak hours.

Compared to the information collected by the techniques above, the image processing method based on bus passenger object detection can provide more information. In recent years, with the continuous upgrading of processor performance, real-time data analyzing based on the image of bus passenger object detection has already been put in use. At present, the techniques of feature extraction and matching recognition based on two-dimensional images have been widely used in bus passenger object detection.

Before 2013, the bus passenger object detection was mainly based on the traditional detection method. That is, the sliding window image traversed the image to detect the position and size of the object in the positioning image^[1]. Moreover, the features in the proposed rectangular regions are extracted to express the characteristics of the region, such as Haar^[2] features commonly used for face detection, HOG^[3] features for

pedestrian detection. Finally, the classifier is used to recognize the object, such as the commonly used classifiers SVM, Adaboost, Decision Tree and so on. However, the disadvantages of traditional detection methods are apparent. Firstly, the region selection strategy dependent on the sliding window is not targeted, with high time complexity and redundant window. Secondly, artificial designing features are not very robust to various changes. In the empirical testing of bus passenger object detection, the average accuracy can only reach 60%-70%. Since 2013, the entire academic community and industry community have gradually tried to use deep learning to realize object detection^[4]. The most typical representative is the R-CNN series (R-CNN, SPPnet, Fast R-CNN, and Faster R-CNN) using the "Proposal + Classification" object detection method as well as the SSD and YOLO methods that directly regress to the detected objects. The object detection method based on deep learning can achieve expected results, and the average accuracy can reach over 95%. However, the overly complicated CNN used in these algorithms is not applicable to the bus passenger object detection with limited resource performance. Thus, these methods are not fit for the bus passenger object detection.

In this paper, we employ the depthwise separable convolution method to optimize the tiny YOLO model so that it can be suitable for the embedded devices in future. We can obtain more effective and accurate statistics via this method, and it will offer more help to the smart public transport system.

II. Related Works

In the image statistics of bus passenger object detection where the image is always moving, the bus passenger object detection speed is so high that it cannot afford slow counting. Otherwise, one person in a frame can be detected, but in the next frame, that person will disappear. Therefore, higher detection speed is required. The detected image is shown as Fig.1.



Fig. 1. The bus passenger object detection results on bus passenger test set

While the CNN model is continually approaching the precision limit of computer vision tasks, its depth and size are also exponentially increasing. What follows is a very awkward scene: such a vast model can only be used on a limited platform, and it cannot be transplanted to mobile

terminals and embedded chips. Even with transmission over the network, the high bandwidth usage is daunting for many users. On the other hand, large-scale models also pose a huge challenge to device power consumption and operating speed.

Currently, mainstream object detection methods based on CNN include Faster R-CNN^[5], SSD^[6], and YOLO^[7]. Based on the regional proposal, the Faster R-CNN utilizes the data-driven approach. Firstly, learn the features through the network. Secondly, propose some proposal regions based on these features. Finally, classify the objects based on these proposal regions.

On the one hand, this method can extract effective proposal regions; on the other hand, shared weight greatly reduces the parameter amount of CNN, which improves the detection speed. The regression-based SSD is a multi-object detection algorithm that directly predicts the object category and the bounding box. The algorithm does not have the process of generating a proposal. It adopts the VGG-19 feature extraction network that deprecates the fully connected layers, so the detection speed is improved. YOLO redesigns the object detection as a single regression problem. It trains on the amplified image and directly optimizes the detection performance to ensure the accuracy. This technique relies on the characteristics of the entire image to predict the position of each bounding box and also predicts which category the bounding box belongs to in the image.

Among these detection methods, the unified model of YOLO has several advantages over others. First of all, YOLO regards the detection as a regression problem. When the detection is performed, CNN directly runs on a new image to predict the detection result without other complicated work, so its detection speed is very fast. Furthermore, YOLO makes global predictions in the image. Unlike techniques based on sliding windows and region proposals, YOLO sees the entire image during training and testing time, so it implicitly encodes the appearance of the class as well as contextual information. YOLO loses half of its background errors. Finally, the generalized representation is good of YOLO learning objects. When training natural images and testing works of art, YOLO is significantly better than one-stage detection methods, and it is unlikely to fail when applied to new domains or accidental inputs.

In the practical bus passenger object detection, the detection object has the characteristics of single detection object and similar size. Here, this paper will use tiny YOLO, a smaller version of YOLO^[8]. It is more suitable for mobile machine learning and IoT devices. The model obtains higher detection accuracy in the task of bus passenger object detection. Besides, unlike the Faster R-CNN and SSD algorithms feature extraction based on the

network structure VGG-16, the network model of tiny YOLO is relatively simple and the detection speed is much faster. As such, tiny YOLO network structure is selected as the research object of the bus passenger object detection.

Though these state-of-the-art object detection networks have achieved high performance on a high-end GPU, these methods are subjected to real-time detection on embedded devices because of their high memory requirement and tremendous FLOPS. Faster R-CNN, a network towards real-time object detection, only has a frame rate of 5fps on GPU, not to mention the use on embedded devices. Lightweight CNN models such as MobileNet^[11], SqueezeNet, and ShuffleNet^[10] are specifically designed for mobile devices. The above network model is used for classification and detection. However, tiny YOLO could be only used for detection. As such, tiny YOLO is more lightweight compared with the above network model. In the bus passenger object detection based on deep learning technology, designing a lightweight network model that can reduce the number of parameters in the CNN model and improve the detection speed to adapt to the object detection task has become an urgent issue to be handled.

III. Tiny YOLO Optimization for Bus Passenger Object Detection

Although tiny YOLO model structure based on CNN can achieve ideal accuracy in bus passenger object detection, the model structure is still too complicated for practical applications of embedded devices with limited resource performance. Moreover, tiny YOLO with the size of 60.5MB is still too implemented on embedded devices although it is reduced considerably compared to YOLO and YOLOv2 (753MB and 193MB, respectively). This paper is aimed at optimizing tiny YOLO model structure based on CNN. By analyzing the time-consuming part of the CNN, a depthwise separable convolution method is proposed in terms of the network structure. While reducing the number of parameters, model size and increasing the speed of detection, it maintains a low loss in detection accuracy.

1. Tiny YOLO network structure

CNN can be simply thought of as being stacked by various building blocks, including convolution, pooling, rectification, and normalization. The function of convolution is to extract features. Pooling is to abstract feature and improves the generalization competence of the model. Rectification is to make the model learn faster and better. Normalization is a nonlinear processing module that can accelerate the training of the model. The complexity of the CNN is mainly manifested in the large amount of calculations caused by the huge number of

parameters, and tiny YOLO is no exception. A network structure table of tiny YOLO is shown in Table 1:

Table 1. Tiny YOLO network structure table

Layer	Name	Filters	Size	Input	Output
0	conv	16	3×3/1	416×416×3	416×416×16
1	max		2×2/2	416×416×16	208×208×16
2	conv	32	3×3/1	208×208×16	208×208×32
3	max		2×2/2	208×208×32	104×104×32
4	conv	64	3×3/1	104×104×32	104×104×64
5	max		2×2/2	104×104×64	52×52×64
6	conv	128	3×3/1	52×52×64	52×52×128
7	max		2×2/2	52×52×128	26×26×128
8	conv	256	3×3/1	26×26×128	26×26×256
9	max		2×2/2	26×26×256	13×13×256
10	conv	512	3×3/1	13×13×256	13×13×512
11	max		2×2/1	13×13×512	13×13×512
12	conv	1024	3×3/1	13×13×512	13×13×1024
13	conv	1024	3×3/1	13×13×1024	13×13×1024
14	conv	30	1×1/1	13×13×1024	13×13×30
15	detection	—	—	—	—

From Table 1, it is indicated that there are more than 15 million convolution kernel parameters in tiny YOLO network structure model. The parameter quantity is mainly related to the number of convolution kernels. The amount of convolution kernel parameters in each layer convolution operation is determined by the number of input feature map channels, the size of the convolution kernel, and the number of convolution kernels. In addition to the input layer in the entire network structure, the number of channels of the convolution kernel in the remaining operation layers is consistent with the number of convolution kernels in the previous layer. The weight file size reflects the parameter amount of the network, and the binary weight file saved by tiny YOLO is 64MB.

This paper tests the object detection performance of tiny YOLO network structure with the help of a bus passenger test set containing 12,749 images. The selected test samples contain various objective natural factors, such as the brightness and darkness of the light^[11] and the influence of different head shapes of the bus passengers on the object detection, as is shown in Fig.2. In this test set, the detection accuracy of tiny YOLO is 0.945 at rush hour.



Fig. 2. Test set image with various objective factors

The GNU profiler tool is used to get the function call relationship of the program at the time of detection. The function call relationship diagram is shown as Fig.3.

It can be seen from Fig.3 that the gemm_nn function responsible for matrix multiplication takes up 86% of

the entire program runtime. This function is called 9 times, corresponding to each convolution operation of 9 convolution layers in the network structure. It can be seen that the excessive number of parameters leads to excessive calculation, which is the main reason of time consumption for the detection model of tiny YOLO.

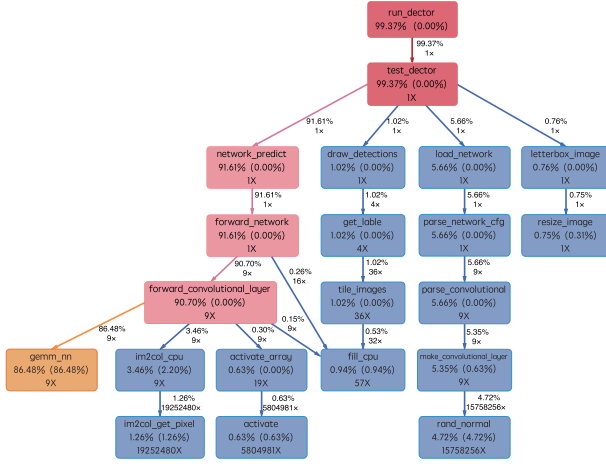


Fig. 3. The function call relationship diagram of tiny YOLO in the test set

2. Optimization of tiny YOLO network structure model by depthwise separable convolution

In order to reduce computational expense of the convolution layer operation for extracting features in the tiny YOLO network, this paper uses depthwise separable convolution method^[12]. It can reduce the convolution kernel parameters and lower the computational cost.

Usually, a conventional convolution operation is a three-dimensional convolution kernel that convolves with an input feature map. Each convolution kernel simultaneously operates each channel of the input feature map, and the channel number of the input feature map is consistent with that of the convolution kernel. If the input tensor of the convolution layer l is $x^l \in R^{H^l \times W^l \times D^l}$, the convolution kernel number of the layer is $f^l \in R^{H \times W \times D^l}$. The three-dimensional input convolution operation simply extends the two-dimensional convolution to all the channels of the corresponding position (*i.e.* D^l), and finally sums all the HWD^l elements processed by once convolution operation as the result of convolution operation in this position. The specific process is shown in Fig.4.

The depthwise separable convolution method is to decompose a complete convolution operation into two steps, namely depthwise convolution and pointwise convolution. Unlike conventional convolution operations, depthwise convolution is a special three-dimensional convolution kernel. A convolution kernel is only responsible for one channel and the number of channels is inconsistent with that of input feature map channels. Depthwise convolution is done entirely in a two-dimensional plane.

The number of convolution kernels is the same as that of channels in the previous layer (the channel and the convolution kernel correspond one-to-one). The pointwise convolution operation is very similar to the conventional convolution operation, whose kernel has a size of $1 \times 1 \times M$, where M is the number of channels of the input feature map. The convolution operation here will handle the maps of the previous step via weighted fusion in the depth direction to generate a new feature map. The number of feature maps is consistent with the number of convolution kernels. The specific process is shown in Fig.5.

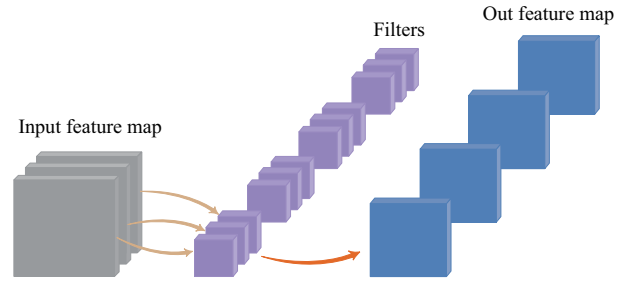


Fig. 4. Conventional convolution operation

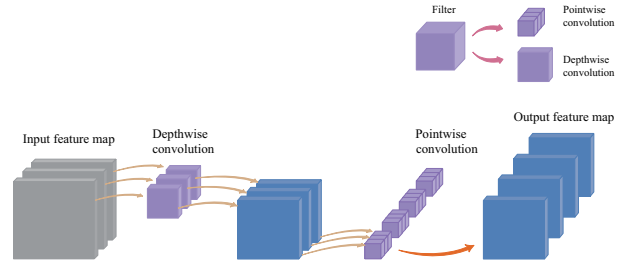


Fig. 5. The depthwise separable convolution

The depthwise separable convolution method divides the original convolutional layer into two convolutional layers. In the depthwise convolution operation, each convolution kernel is convolved with each channel of the input only. The pointwise convolution operation is responsible for feature fusion that combines the results of the previous convolution. The original three-dimensional convolution kernel is a convolution operation between multiple input channels and convolution kernels to get an output channel. After the improvement, only one channel is required to obtain an output channel, and then a 1×1 convolution is used for channel feature fusion. The advantage of this method is the reduction of parameters and their calculation. The number of parameters of depthwise separable convolution is about 1/3 of the conventional convolution, which can reduce the calculation by about 8 to 9 times.

In the object detection of bus passenger, the lightweight network model of tiny YOLO improved by the depthwise separable convolution method is designed for embedded with more suitability. The original standard

convolution operation will be split into two parts: depthwise convolution and pointwise convolution. The network structure modification method is shown in Fig.6.

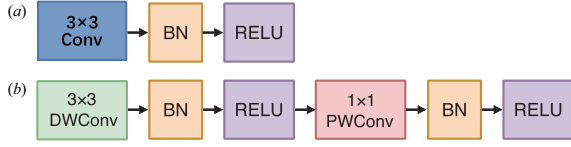


Fig. 6. Tiny YOLO model improvement method. (a) Standard convolution; (b) Depthwise separable convolution

In this paper, several experimental groups were set up for comparative analysis to fully map out the improvement of computational efficiency by reducing the parameter via depthwise separable convolution method.

Tiny YOLO network model has a total of 9 convolutional layers.

The experiment gradually improved 7 convolution layers except the first and the last in tiny YOLO network model so as to gradually replace each layer of standard convolution with depthwise convolution and pointwise convolution. The seven improved network models are named M_1 , M_2 , M_3 , M_4 , M_5 , M_6 and M_7 respectively, with the only difference lying in the convolutional layer. This step-by-step improvement is more conducive to the intuitive and clear observation of the increase in the speed of the reducing parameters in the base layer of each volume and in each convolutional layer. The network model structure is shown in Table 2.

Table 2. Improved network model configuration

M_1	M_2	M_3	M_4	M_5	M_6	M_7
10 layers	11 layers	12 layers	13 layers	14 layers	15 layers	16 layers
Input(416×416 RGB figure)						
conv3-16	conv3-16	conv3-16	conv3-16	conv3-16	conv3-16	conv3-16
max_pool 2/2						
conv3-32	conv3-32	conv3-32	conv3-32	conv3-32	conv3-32	conv3dw-16 conv1-32
max_pool 2/2						
conv3-64	conv3-64	conv3-64	conv3-64	conv3-64	conv3dw-32 conv1-64	conv3dw-32 conv1-64
max_pool 2/2						
conv3-128	conv3-128	conv3-128	conv3-128	conv3dw-64 conv1-128	conv3dw-64 conv1-128	conv3dw-64 conv1-128
max_pool 2/2						
conv3-256	conv3-256	conv3-256	conv3dw-128 conv1-256	conv3dw-128 conv1-256	conv3dw-128 conv1-256	conv3dw-128 conv1-256
max_pool 2/2						
conv3-512	conv3-512	conv3dw-256 conv1-512	conv3dw-256 conv1-512	conv3dw-256 conv1-512	conv3dw-256 conv1-512	conv3dw-256 conv1-512
max_pool 2/1						
conv3-512	conv3dw-512 conv1-1024	conv3dw-512 conv1-1024	conv3dw-512 conv1-1024	conv3dw-512 conv1-1024	conv3dw-512 conv1-1024	conv3dw-512 conv1-1024
conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024	conv3dw-1024 conv1-1024
conv1-30						
Detection						

IV. Experimental Evaluation

1. Experimental hardware environment and dataset

As shown in Table 3, the detailed information of the operating system, CPU, and RAM used in this experiment.

The passenger dataset in this paper consists of 12,749 images captured by bus videos from the camera photographing bus passengers vertically. The ratio of training set and test set in these images is approximately 4:1. They are about different weathers in our bus videos, including sunny day, rainy day, snowy day and so on. As a result of driving bus in day and night, light condition is

a factor that cannot be ignored that often causes errors in the bus passenger object detection. Experiment has prepared enough data set to take everything into account that can take place in the bus to ensure the reliability of our data.

Table 3. Experimental hardware environment parameter

Name	Type and configuration parameters
Operation system	Ubuntu 16.0.4
CPU	Intel(R) Core(TM) i7-7700K CPU @ 4.20GHz
RAM	DDR4 4×8G

To create annotations, we have manually annotated tracks of human upper-body in these images. For each upper-body track, we use the upper-body bounding

box^[13]—the smallest axis parallel rectangles including all visible pixels of the upper-body to represent one bus passenger. According to the above step-by-step optimization model, seven sets of comparative experiments were designed. The hyper-parameters of the network structure (excluding the number of convolution kernels) were taken consistent default settings in the seven experiments. Some hyper-parameters are shown in Table 4.

2. Analysis of experimental results

In seven sets of contrast experiments, this paper describes and measures the original network model and

improved network model from the aspects of Average precision(AP), time consuming, Frames per second(FPS), and model weight size, as shown in Table 5.

In terms of AP, the values of each network model differ slightly, and the value that varies greatly is just declining about one percentage. It can be seen from the time-consuming detection, the time for detecting an image in the improved network model is reduced from 0.915s to 0.225s, compared with the original network model. In terms of FPS, the detection speed increases from 1.093 frames to 4.449 frames, with an obvious improvement in the detection speed. In terms of weight file size, it is

Table 4. Hyper-parametric configuration table

Interactive times	Learning rate	Input size	Momentum	Weight decay	Training algorithm	Batch
42000	0.001	416×416×3	0.9	0.0005	gradient descent	64

Table 5. Performance of each improved network

Network	AP	Time consuming(s)	FPS	Weight size(MB)
tiny YOLO	0.945	0.915	1.093	60.5
M ₁	0.945	0.583	1.715	29
M ₂	0.937	0.401	2.494	13
M ₃	0.942	0.384	2.607	8.3
M ₄	0.940	0.370	2.703	7.3
M ₅	0.935	0.319	3.131	7.1
M ₆	0.938	0.271	3.686	7
M ₇	0.936	0.225	4.449	7

reduced from the original 61MB to 7MB. The network model M₇ improved by 7 standard convolutional layers is compared with tiny YOLO network model in the passenger object detection task. The contrast is shown in Fig.7.

It can be found from Fig.7: in four different scenes, both tiny YOLO and the improved M₇ network model can detect and identify all passengers well, and there is no difference in the number of detections.

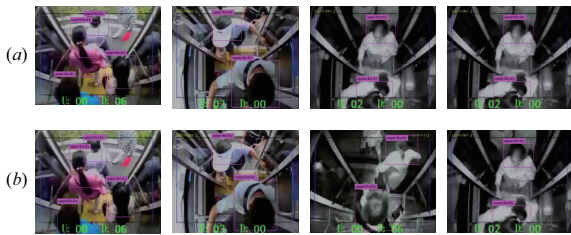


Fig. 7. Comparison of tiny YOLO and M₇ object detection effects. (a) Tiny YOLO detection effect; (b) M₇ detection effect

Analyzed from the perspective of the recall rate accuracy, Fig.8 reveals the recall accuracy curve figure of the M₇ and tiny YOLO network structures. In the Fig.8, the abscissa is the recall rate and the ordinate is the detection accuracy. From the figure, it can be seen that the detection accuracy of each network model drops significantly when the recall rate exceeds 0.93. In the case

of low recall rate, the improved network model has higher detection accuracy than the original network model.

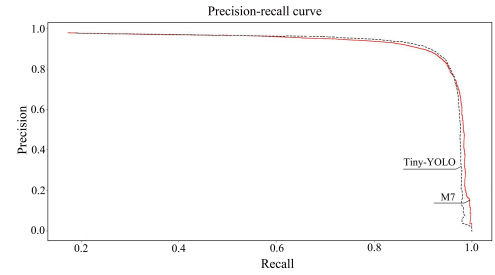


Fig. 8. Precision-Recall curve of different network model

Based on the indicators above, it can be seen that the depthwise separable convolution method used to improve tiny YOLO network model can significantly improve the detection speed while reducing the detection time as well as weight file size. In particular, there is almost no sacrifice in detection accuracy.

V. Conclusions

The detection object has the characteristics of single detection object and similar size in the actual scene of bus passenger object detection. Compared with the current mainstream object detection method, the lightweight network mode tiny YOLO is selected with reference to the index of the detection speed and the detection accuracy. Meanwhile, in order to further improve tiny YOLO network model with the drawbacks of too many parameters, this paper uses a depthwise separable convolution method to optimize tiny YOLO network model, so that it can be applied to embedded system and mobile terminals. The depthwise separable convolution method was then verified by setting up seven sets of experiments. The experimental results show that after seven layers of convolution replacement, the detection speed is significantly improved, and the

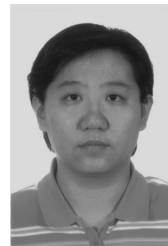
detection accuracy is almost the same with that of the original network model. The improved model can achieve good detection results for bus passenger object detection and is robust enough to be applied to embedded devices.

References

- [1] R. Girshick, J. Donahue, T. Darrell, *et al.*, "Rich feature hierarchies for accurate object detection and semantic segmentation", *The IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, United States, Columbus, U.S., pp.580–587, 2014.
- [2] P. Viola and M.J. Jones, "Robust real-time face detection", *International Journal of Computer Vision*, Vol.57, No.2, pp.137–154, 2004.
- [3] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection", *Computer Vision and Pattern Recognition*, San Diego, United States, pp.886–893, 2005.
- [4] J. Huang, V. Rathod, C. Sun, *et al.*, "Speed/accuracy trade-offs for modern convolutional object detectors", *The IEEE Conference on Computer Vision and Pattern Recognition*, Hawaii, U.S., pp.7310–7311, 2017.
- [5] S.Q. Ren, K.M. He, R. Girshick, "Faster r-cnn: Towards real-time object detection with region proposal networks", *Advances in Neural Information Processing Systems 28*, Montreal, Quebec, Canada, pp.91–99, 2015.
- [6] W. Liu, D. Anguelov D. Erhan, *et al.*, "SSD: Single shot multiBox detector", *European Conference on Computer Vision*, Amsterdam, The Netherlands, pp.21–37, 2016.
- [7] J. Redmon, S. Divvala, R. Girshick, *et al.*, "You only look once: Unified, real-Time object detection", *The IEEE Conference on Computer Vision and Pattern Recognition*, Las Vegas, U.S., pp.779–788, 2016.
- [8] J. Redmon, "YOLO: Real-time object detection", <https://pjreddie.com/darknet/YOLO/>, 2018-3-25.
- [9] C.E. Perez, "Building a 50 Teraflops AMD Vega Deep Learning Box for Under \$3K", <https://medium.com/intuition-machine/building-a-50-teraflops-amd-vege-deep-learning-box-for-under-3k-ebdd60d4a93c>, 2017-7-14.
- [10] X.Y. Zhang, X.Y. Zhou, M.X. Lin, *et al.*, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices", *The IEEE Conference on Computer Vision and Pattern Recognition*, Salt Lake City, U.S., pp.6848–6856, 2018.
- [11] G.Y. Chen, S. Krishnan, Y.J. Zhao, *et al.*, "Illumination invariant face recognition", *ICIC 2013: Intelligent Computing Theories*, Nanning, China, pp.385–391, 2013.
- [12] L. Sifre and S. Mallat, "Rigid-motion scattering for image classification", *Ph.D.Thesis*, Ecole Polytechnique, CMAP, France, 2014.
- [13] M. Mathias, R. Benenson, M. Pedersoli, *et al.*, "Face detection without bells and whistles", *European Conference on Computer Vision*, Zurich, Switzerland, pp.720–735, 2014.



ZHANG Shuo was born in 1989. He received the B.S. and M.S. degrees in computer science from Harbin Engineering University. He is now a Ph.D. candidate of Harbin Engineering University. His research interests include embedded system and information security. (Email: 49866028@qq.com)



architecture. (Email: wuyanxia@hrbeu.edu.cn)

WU Yanxia (corresponding author) was born in 1979. She received the B.S., M.S. and Ph.D. degrees from Harbin Engineering University, Harbin, China. Now she is an associate professor in the College of Computer Science and Technology in Harbin Engineering University. Her current research interests include compiler technology and computer



processing and information security.

MEN Chaoguang was born in 1963. He received the B.S. and Ph.D. degrees at School of Computer Science and Technology in Harbin Institute of Technology in 1985 and 2004 respectively. He is now a professor in the College of Computer Science and Technology from Harbin Engineering University, Harbin, China. His research is focused on image



LI Xiaosong was born in 1993. He received the B.S degree in computer technology from Harbin Engineering University, Harbin, China, in 2018. He is currently pursuing the Ph.D. degree majoring in software engineering. His research interests include computer vision, image processing, machine learning and deep learning.